Semantic Log Analytics for UAV Flight Logs

A concept explainer for the Advanced Database Management Systems term project

Sumit Kumar (22CS30056)

Abstract

This document explains every idea in the project to a reader who has not seen it. Each section gives the plain-English story first and the technical detail (and the measured numbers) second. All figures come from the saved result files under `bench/results/`, and each design decision is logged with its evidence in `docs/decisions.md`.

Contents

1 The Big Picture **2**

2 Core Concepts **2**
2.1 Time series, hypertables and chunks . . . 2
2.2 Columnstore and compression . . . 3
2.3 Embeddings and nearest-neighbour search . . . 3
2.4 Reciprocal rank fusion . . . 3
2.5 An ontology as relational tables . . . 3
2.6 Job queues in a database . . . 3

3 Contribution 1: Schema and the narrow telemetry table **4**

4 Contribution 2: The semantic layer **4**

5 Contribution 3: Pipelines that survive failure **5**

6 Contribution 4: Hybrid search and the self-routing planner **6**

7 Contribution 5: Guarded natural-language SQL **7**

8 Results **8**
8.1 Storage . . . 8
8.2 Columnstore design experiment . . . 8
8.3 TimescaleDB against DuckDB . . . 8
8.4 Vector indexes . . . 8
8.5 Filtered search . . . 9
8.6 Search quality and natural-language SQL . . . 9
8.7 The application . . . 9

9 Theoretical Comparison Table **9**

10 Glossary **9**

11 Frequently Asked Questions **10**

12 Conclusions **11**

1 The Big Picture

A drone writes a *flight log* on every flight: positions, attitudes, battery voltage, motor commands, plus text messages such as warnings. One log is easy to look at. A *collection* of logs is hard, because the data is three different things at once: a very dense time series (the project corpus has 70.7 million samples from only 300 flights), free text (3,401 events), and vendor-specific names (PX4 and ArduPilot call the same quantity different things).

Key concept

Thesis. Treat the whole log collection as a database problem. One PostgreSQL instance stores the time series compactly, maps vendor fields to shared meanings, searches events by meaning, and lets a language model answer questions in SQL under strict safety rules. Every design choice is justified by a measurement.

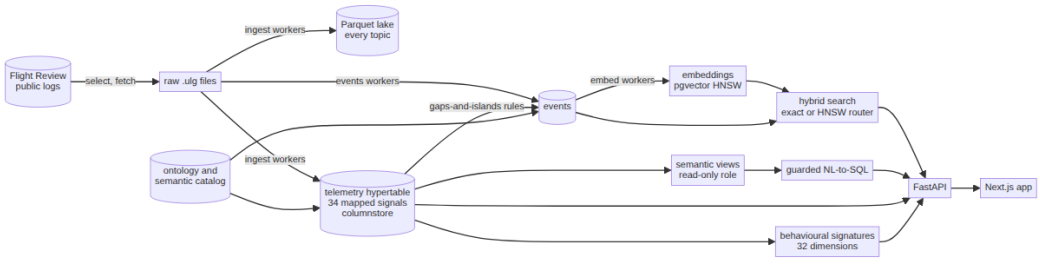


Figure 1: Architecture: raw logs become a Parquet lake, the canonical telemetry table and events; embeddings and the ontology sit beside them; the application reaches data only through search and guarded read-only views.

The work answers four questions: (1) how to store high-rate telemetry and at what cost, (2) which vector index and filtering strategy retrieve events best, (3) whether a model can write safe and correct SQL and what a semantic layer adds, and (4) whether the pipeline survives failure and scales.

Insight

The project reports negative results as carefully as positive ones: hybrid text-plus-vector search did *not* beat vector-only search, and the semantic views' gain in NL-to-SQL accuracy was *not* statistically significant. Saying so is part of the method.

2 Core Concepts

2.1 Time series, hypertables and chunks

Plain English: a time series is a list of measurements stamped with a time. A **hypertable** (TimescaleDB) looks like one table but is silently cut into *chunks*, one per time range, so recent data stays small and old data can be treated differently.

Definition

Hypertable: a logical table partitioned by time into chunks. Here, telemetry uses 7-day chunks.

2.2 Columnstore and compression

Plain English: a normal table stores row by row. A columnstore stores each column together, so similar numbers sit side by side and compress extremely well. Reads that need only a few columns, or a few signals, skip the rest.

Definition

Columnstore with `segmentby`: compressed chunk storage where rows are grouped by the `segmentby` columns and ordered by `orderby`. Here the grouping is (flight, signal), ordered by time.

2.3 Embeddings and nearest-neighbour search

Plain English: an embedding turns text into a list of 1,536 numbers so that texts with similar meaning land near each other. Searching by meaning means finding the nearest points. Comparing against every point is exact but slow, so *approximate* indexes trade a little accuracy for speed.

- **HNSW**: a layered graph; you hop from node to nearby node toward the query.
- **IVFFlat**: points are sorted into clusters; you search only the closest few clusters.
- **DiskANN**: a compressed graph built to live mostly on disk (pgvectorscale).

Math: recall@k is the fraction of the true top-k neighbours the index returns, recall@$k = |A \cap T|/k$ with A the approximate and T the exact result.

2.4 Reciprocal rank fusion

Plain English: when two rankers disagree (vector similarity vs keyword match), fuse them by rank, not by raw score, because their scores are not comparable. *Math:*

$$\mathrm{RRF}(d) = \sum_i \frac{w_i}{k + \mathrm{rank}_i(d)}$$

with weights w_i and a smoothing constant k.

2.5 An ontology as relational tables

Plain English: an ontology is a dictionary of concepts and how they relate. Here it is ordinary tables (concept, relation, vendor field, field mapping) walked with recursive queries.

2.6 Job queues in a database

Plain English: instead of a separate queue service, the work list lives in a Postgres table. Workers take items with `FOR UPDATE SKIP LOCKED`, which means "take a row nobody else holds".

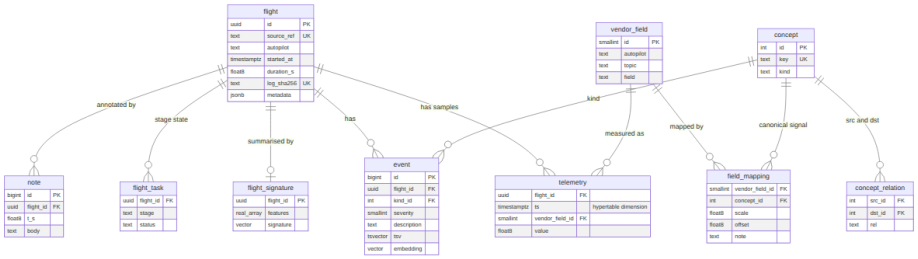


Figure 2: Entity-relationship overview.

3 Contribution 1: Schema and the narrow telemetry table

Plain English: instead of one column per signal, telemetry has one row per (time, flight, field). It is like a ledger: new kinds of expense add rows, never columns. A new signal, or a whole new autopilot, needs no schema change.

Migration 0001: the telemetry hypertable

```
79      # ---- Telemetry: only ontology-mapped ("hot") fields; full-rate raw stays in Parquet ----
80      op.execute("""
81      CREATE TABLE telemetry (
82          flight_id       uuid     NOT NULL REFERENCES flight(id) ON DELETE CASCADE,
83          ts              timestamptz NOT NULL,
84          vendor_field_id smallint NOT NULL REFERENCES vendor_field(id),
85          value           double precision NOT NULL
86      )""")
87      op.execute("""
88      SELECT create_hypertable('telemetry', by_range('ts', INTERVAL '7 days'))
89      """)
90      op.execute("CREATE INDEX telemetry_flight_field_ts ON telemetry (flight_id, vendor_field_id, ts DESC)")
91
92      # ---- Semantic view: telemetry expressed in canonical concepts ----
93      op.execute("""
94      CREATE VIEW canonical_telemetry AS
```

Found by testing

As a plain rowstore the index (7.6 GB) was larger than the table (4.9 GB). That observation is why the columnstore work matters.

4 Contribution 2: The semantic layer

Plain English: PX4 might call battery voltage `battery_status.voltage_v`; ArduPilot calls it something else. The ontology gives both a shared name, `battery.voltage`, plus a scale, an offset and a note describing the conversion.

Definition

Semantic layer: 63 concepts and 55 relations (34 signals, 7 subsystems, 7 failure modes, 15 event kinds) plus a field-mapping table, exposed through documented read-only views.

Insight

Five ArduPilot groups were deliberately *not* mapped (vibration, motor outputs, EKF ratios, barometric altitude, flight mode) because their meanings differ. A wrong mapping produces confident wrong answers; a gap produces an honest "not available".

ArduPilot mappings carry a scale, an offset and a semantic note

```
ARDUPILOT_FIELDS = [
    ("ATTITUDE", "roll", "rad", "attitude.roll", RAD2DEG, 0, "radians to degrees"),
    ("ATTITUDE", "pitch", "rad", "attitude.pitch", RAD2DEG, 0, "radians to degrees"),
    ("ATTITUDE", "yaw", "rad", "attitude.yaw", RAD2DEG, 0, "radians to degrees, range -180 to 180"),
    ("GPS_RAW_INT", "lat", "degE7", "gps.lat", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
    ("GPS_RAW_INT", "lon", "degE7", "gps.lon", 1e-7, 0, "degrees times 1e7; 0 means no fix and is dropped"),
    ("GPS_RAW_INT", "alt", "mm", "gps.alt_msl", 0.001, 0, "millimetres to metres, above mean sea level"),
    ("GPS_RAW_INT", "fix_type", "enum", "gps.fix_type", 1, 0, "the same MAVLink enum PX4 uses (3 is a 3D fix)"),
    ("GPS_RAW_INT", "satellites_visible", "count", "gps.satellites", 1, 0, "satellites VISIBLE; PX4 reports satellites USED, so counts run slight
    ("VFR_HUD", "groundspeed", "m/s", "gps.speed", 1, 0, "ground speed from the navigation filter; PX4 reports GNSS speed"),
    ("LOCAL_POSITION_NED", "x", "m", "position.x", 1, 0, "local north, relative to home"),
    ("LOCAL_POSITION_NED", "y", "m", "position.y", 1, 0, "local east, relative to home"),
    ("LOCAL_POSITION_NED", "z", "m", "position.z", 1, 0, "local down, relative to home"),
    ("LOCAL_POSITION_NED", "vx", "m/s", "velocity.x", 1, 0, "north"),
```

Events. Events come from log transitions, the failure detector, and *gaps-and-islands* rules over telemetry (window functions that find runs of consecutive abnormal rows). Rules fire only relative to a baseline in the same flight, and near-duplicate messages collapse (3,846 to 3,401).

5 Contribution 3: Pipelines that survive failure

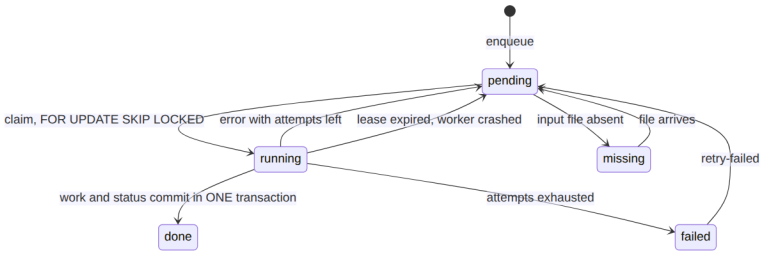


Figure 3: States of a job item.

Plain English: a deli counter with a ticket machine. Each worker takes a ticket, nobody serves the same ticket twice, and if a worker disappears the ticket returns to the pile when its lease expires.

Key concept

Every batch stage is idempotent, tracked in a job table (status, attempts, last error), claimed with `SKIP LOCKED`, failure-isolated, retried, resumable and atomic per item (data and status commit together). To scale you enqueue more items; you do not change code.

Job queue claim: FOR UPDATE SKIP LOCKED in a materialised CTE

```
# MATERIALIZED matters: without it Postgres may re-run the LIMIT .. FOR UPDATE SKIP LOCKED subquery inside the
# join and claim more rows than the limit.
CLAIM = """
WITH c AS MATERIALIZED (
    SELECT flight_id FROM flight_task
    WHERE stage=%(stage)s AND attempts < %(max)s
      AND (status='pending' OR (status='running' AND claimed_at < now() - make_interval(mins => %(lease)s)))
    ORDER BY created_at, flight_id LIMIT %(n)s FOR UPDATE SKIP LOCKED)
UPDATE flight_task t SET status='running', attempts=attempts+1, claimed_at=now(), worker=%(worker)s
FROM c WHERE t.flight_id = c.flight_id AND t.stage = %(stage)s
RETURNING t.flight_id"""


def enqueue(conn, stage: str) -> int:
```

Found by testing

`SKIP LOCKED` once claimed 52 rows when asked for 25, because the planner re-ran the subquery. The fix is a `MATERIALIZED` CTE. A second bug: a transaction block opened after a read became a *savepoint*, so nothing committed while the run reported success. Both were found by testing from a fresh connection.

6 Contribution 4: Hybrid search and the self-routing planner

Plain English: searching events combines meaning (vector), keywords (full text), and hard filters (kind, flight, time). The trap is that pgvector applies a filter *after* it finds neighbours. If a filter keeps one row in a thousand, the index may find 40 neighbours and none pass.

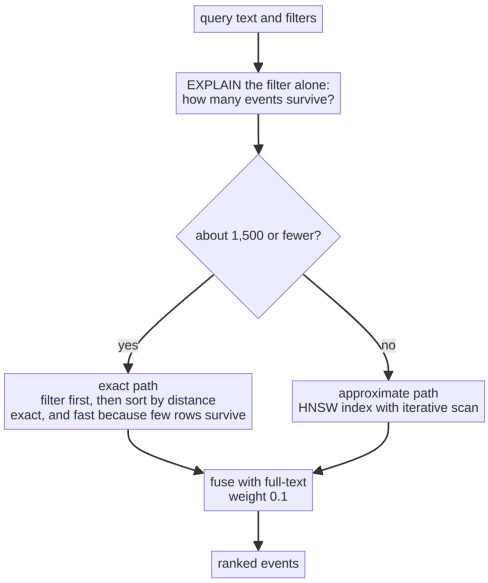


Figure 4: Router: estimate the filter's rows with `EXPLAIN`; exact path if about 1,500 rows or fewer, HNSW with iterative scan otherwise.

Insight

With both indexes present and a filter keeping 10% of rows, the planner chose HNSW post-filtering: it returned **4.0 of 10 rows** with recall 0.37, in 1.6 ms. It is fast and wrong, because a cost model estimates time, not completeness. The router avoids depending on that choice.

7 Contribution 5: Guarded natural-language SQL

Plain English: a model that writes SQL could, if tricked, read files or run a query that takes the server down. "It only runs SELECT" is not a safety boundary. The design uses four independent layers so that defeating one is not enough.

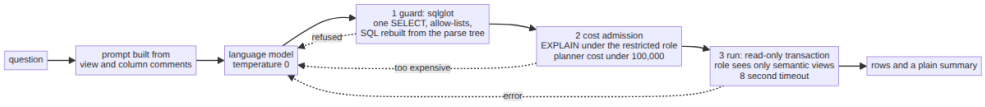


Figure 5: Guard pipeline.

1. **Guard** (`sqlglot`): exactly one SELECT, allow-listed tables and functions, no nested or recursive WITH; the SQL that runs is *regenerated from the parse tree*, so what was checked is what runs.
2. **Role**: a database role that can read only the documented views.
3. **Read-only transaction** with an 8 s statement timeout.
4. **Cost admission**: `EXPLAIN` first; refuse when the planner's estimate exceeds 100,000.

SQL guard: one SELECT, no forbidden statements, top-level WITH only

```
    root = statements[0]
    if not isinstance(root, (exp.Select, exp.SetOperation)):
        raise GuardError(f"Only SELECT queries are allowed, got {type(root).__name__.upper()}.")

    bad = next((n for n in root.walk() if isinstance(n, FORBIDDEN_NODES)), None)
    if bad is not None:
        raise GuardError(f"{type(bad).__name__.upper()} is not allowed in a query.")

    withs = [n for n in root.walk() if isinstance(n, exp.With)]
    if any(w is not root.args.get("with_") and w is not root.args.get("with") for w in withs):
        raise GuardError("WITH is only allowed at the top level of the query.")
    if any(w.args.get("recursive") for w in withs):
        raise GuardError("Recursive WITH is not allowed.")
    cte_names = {c.alias_or_name.lower() for c in root.find_all(exp.CTE)}

    used_tables: list[str] = []
    for t in root.find_all(exp.Table):
        if not isinstance(t.this, exp.Identifier):
            raise GuardError("Table-valued functions are not allowed in FROM.")
        name, schema = t.name.lower(), (t.db or "").lower()
```

Key concept

46 hostile queries were each refused *for the intended reason.* Realistic questions plan under 5,000, while a telemetry self-join plans at 515 million and is refused before running.

8 Results

8.1 Storage

Representation (70.7 M rows)	Size	Bytes per row
PostgreSQL rowstore + index	12,459 MB	176.2
TimescaleDB columnstore	385 MB	5.4
Parquet (zstd, sorted)	392 MB	5.5
DuckDB native file	700 MB	9.9

Table 1: The columnstore is 32× smaller than the rowstore, and results matched the Parquet source.

8.2 Columnstore design experiment

Variant	Size	Q1	Q2	Q3	Q4
rowstore + B-tree	3,631 MB	10.8 ms	1,519 ms	65 ms	246 ms
segment (flight, signal), order ts	147 MB	12.3	16	10.8	2.7
segment flight only	149 MB	12.9	18.0	11.6	3.2
segment signal only	154 MB	13.0	20.3	14.1	2.2
no segmentby	157 MB	15.0	29.7	13.7	3.6
1-day chunks	150 MB	16.3	15.3	12.3	3.4
30-day chunks	143 MB	9.2	8.3	5.7	2.4

Table 2: 100 flights, 26.9 M rows, median of 7 warm runs. Q1 one signal of one flight; Q2 one signal across all flights; Q3 every signal of one flight; Q4 one signal in a 90-day window.

8.3 TimescaleDB against DuckDB

Query (p50, ms)	Timescale columnstore	DuckDB over Parquet	DuckDB native
Q1 one signal, one flight, 1 s buckets	25.4	25.3	4.9
Q2 one signal, all flights	69.2	30.6	12.9
Q3 every signal of one flight	18.7	45.7	22.5
Q4 one signal, 90-day window	3.8	28.4	4.5
Q5 count and mean per signal, all data	811.8	1,057.8	478.3
Q6 min/max/mean per flight and signal	1,007.5	2,605.3	2,004.0

Table 3: 70,731,891 rows, median of 5 warm runs. Load: Timescale `COPY` 286,792 rows/s; DuckDB bulk insert from Parquet 6,153,564 rows/s; DuckDB row-by-row insert 603 rows/s.

8.4 Vector indexes

A controlled 50,000-vector comparison at two noise levels (relative contrast 6.0 and 2.0, the real corpus being 20.5) left the ranking unchanged: HNSW m=32 was fastest at recall 0.99 in both,

Index (100,000 vectors)	Build	Size	p50 at recall ≥ 0.95	≥ 0.99
HNSW m=16	56 s	819 MB	2.7 ms	not reached
HNSW m=32	149 s	819 MB	2.4 ms	3.2 ms
IVFFlat, 316 lists	17 s	822 MB	5.5 ms	9.3 ms
IVFFlat, 100 lists	8.5 s	820 MB	8.4 ms	14.8 ms
DiskANN	268 s	68 MB	not reached	not reached (max 0.74)

Table 4: 1,536 dimensions, 200 queries, top 10; exact search takes 362 ms.

and DiskANN peaked at 0.845 and 0.889 recall.

8.5 Filtered search

Strategy (rows returned of 10)	keeps 0.1%	1%	10%	50%
HNSW post-filter, ef 40	0.0	0.4	4.1	10
HNSW post-filter, ef 400	0.3	4.2	10	10
HNSW iterative scan	10	10	10	10
B-tree filter, exact sort	10	10	10	10

Table 5: Exact sort takes 1.0, 7.0, 55 and 329 ms at the four selectivities; iterative scan takes 50, 6.8, 2.3 and 1.8 ms. Crossover near 1,000 rows.

8.6 Search quality and natural-language SQL

Search method (65 queries)	nDCG@10	hard paraphrases
vector only	0.710	0.59
vector + light full text (0.1)	0.709	0.59
plain RRF	0.600	0.43
full text only	0.48	0.31

Table 6: Plain fusion is worse than vector only: paired difference -0.109, 95% CI $[-0.172, -0.051]$.

Insight

Without documentation the model rarely failed loudly: 17 of 27 answers were confident and wrong. It never fabricated an answer to an unanswerable question.

8.7 The application

9 Theoretical Comparison Table

10 Glossary

Hypertable Time-partitioned table in TimescaleDB.

Chunk One time-range partition of a hypertable.

Columnstore Compressed column-wise storage of old chunks.

Continuous aggregate Incrementally maintained rollup (here 1-second min, max, mean).

NL-to-SQL condition (27 held-out questions)	Strict execution accuracy
semantic views	0.89
raw tables + same documentation	0.70
raw tables alone	0.26

Table 7: Documentation: 0.26 to 0.70, 12 questions gained and none lost ($p = 0.0005$). Views add 0.19 more (6 gained, 1 lost, $p = 0.125$, not significant).

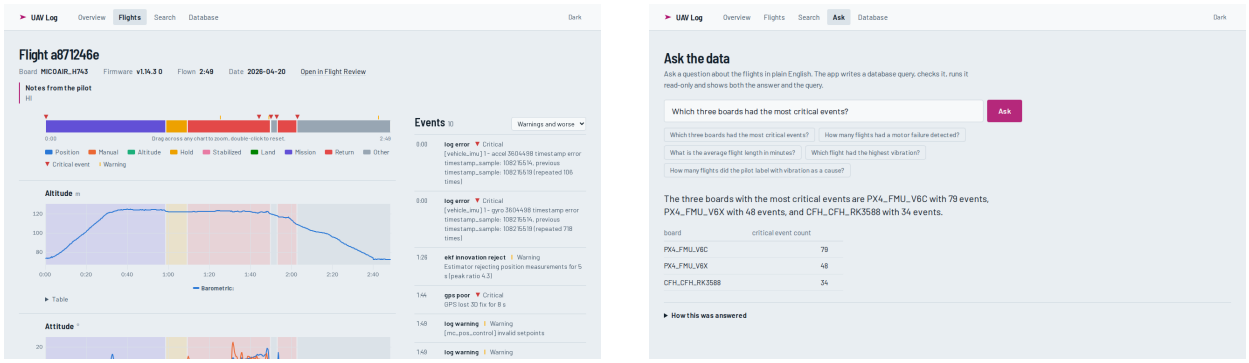


Figure 6: A flight (mode ribbon, min/max envelopes, events) and the Ask page (generated SQL, cost, guard actions).

Embedding A list of numbers representing meaning; here 1,536 dimensions for events, 32 for flight signatures.

HNSW / IVFFlat / DiskANN Approximate nearest-neighbour index families.

Recall@k Fraction of the true top-k returned.

nDCG@10 Ranking quality over the top 10, rewarding relevant results near the top.

RRF Reciprocal rank fusion of several rankings.

Ontology Concepts and relations; here stored in relational tables.

Gaps and islands SQL pattern that finds runs of consecutive rows.

SKIP LOCKED Row-locking mode that skips rows another transaction holds.

Idempotent Safe to run twice with the same result.

Execution accuracy (EX) Share of generated queries whose result equals the gold result.

RC10 Relative contrast of the 10th neighbour; lower means harder data.

Parquet Open columnar file format.

11 Frequently Asked Questions

Why one database instead of several specialised ones? Transactions, roles, joins to the ontology and vectors then live together. DuckDB won some scans in the comparison but gives up concurrent writers and roles.

Is 300 flights enough? It is a stated limit. The storage results are measured on 70.7 million rows; vector benchmarks use 50,000 and 100,000 perturbed vectors. A large run on 10,000 or more logs is planned and every volume is configurable.

Choice	Strength	Weakness	Used here for
Rowstore + B-tree	Fast point reads, simple	176 B/row, slow scans	Baseline
Columnstore	32× smaller, pruned reads	Updates cost more	Telemetry
DuckDB / Parquet	Fast scans, no server	Single writer, no roles	Comparator
HNSW	Fastest queries at high recall	Slow build, large	Event search
IVFFlat	Fast build	Slower queries	Comparator
DiskANN	12× smaller	Lower recall here	Comparator
Post-filter	Simple	Under-returns	Avoided
Iterative scan	Restores row count	50 ms at 0.1%	Large filters
Exact sort	Exact, cheap if selective	Linear in matches	Selective filters

Table 8: Design alternatives and where each was used.

Why did hybrid search lose? On 65 queries vector similarity already handles paraphrases, and plain fusion lets weak keyword hits displace good vector hits. With a small keyword weight it ties; no evidence was found that it helps.

Is the NL-to-SQL "safe"? It is defended in four independent layers and tested with 46 hostile queries. The remaining risk is correctness (confident wrong answers), which is why the SQL and cost are shown.

What is untested? The snapshot/restore script was deliberately left untested and is documented that way, and the large-scale run has not happened.

12 Conclusions

- A single PostgreSQL instance can hold the time series, the ontology, the vectors and a guarded natural-language interface.
- Physical design reduced telemetry from 12.5 GB to 385 MB (32×); measured, not assumed.
- For retrieval, filtering strategy mattered more than index choice, and a self-routing query beat any fixed plan.
- Documentation, more than views, drives NL-to-SQL accuracy; four independent layers make it safe.
- Limits: 300 flights, one machine, one model, author-written questions. Next: the large run, ArduPilot at scale, a concurrency benchmark.